

MrBERT Explainer: What We Changed and Why

Adapted from [MrT5: Dynamic Token Merging for Efficient Byte-level Language Models](#)
(Kallini et al., 2024)

This document explains every modification made to standard BERT to create MrBERT, with the intuition behind each change and pointers to the relevant code. MrBERT adapts the delete gate mechanism from MrT5 to the BERT architecture.

Contents

1	The Core Idea	1
2	Config Changes	2
3	The Delete Gate	2
3.1	The ScaledSigmoid Activation (Lines 160–169)	2
3.2	The SigmoidDeleteGate Module (Lines 178–249)	3
4	Soft Deletion: Masking Attention	3
4.1	How It Works (Lines 414–419)	3
4.2	Softmax1 Variant (Lines 422–425)	4
5	Hard Deletion: Physically Removing Tokens	4
5.1	Step 1: Decide Which Tokens Survive (Lines 509–541)	4
5.2	Step 2: Gather Surviving Tokens (Lines 543–553)	4
6	Protecting Special Tokens	4
7	Where the Gate Sits in the Encoder	5
7.1	Forward Pass Gate Propagation (Lines 670–707)	5
8	Training: The Deletion Loss and PI Controller	5
8.1	The Deletion Loss (Lines 347–387)	5
8.2	The PI Controller (Lines 303–344)	5
8.3	The Regularizer Delay (Lines 977–985)	6
8.4	Separate Learning Rate for the Gate (Lines 850–866)	6
9	Gate Variants	6
10	What Stays the Same	6
11	End-to-End Flow	7
12	Key File Reference	8

1 The Core Idea

Standard BERT processes every token through all 12 encoder layers, even tokens that carry little semantic information (e.g. punctuation, filler words, repeated content). This is computationally wasteful.

The MrBERT idea: after a chosen encoder layer, assign each token a score. Low-scoring tokens are either ignored by subsequent attention layers (soft deletion) or physically removed from the sequence (hard deletion). The rest of the encoder only processes the surviving tokens.

This is a learned compression: the model is trained to identify and discard tokens that don't contribute to the task, reducing the effective sequence length and therefore the compute required for the remaining layers.

2 Config Changes

File: `mrbert/models/configuration_mrbert.py`, lines 48–71

`MrBertConfig` extends the standard `BertConfig` with the following new fields:

Parameter	Default	Purpose
<code>delete_gate_layer</code>	3	Which encoder layer emits the gate (0-indexed). Layers after this one only see surviving tokens.
<code>deletion_type</code>	"scaled_sigmoid"	Which gate variant to use.
<code>sigmoid_mask_scale</code>	-30.0	Controls the strength of the deletion signal added to attention scores.
<code>deletion_threshold</code>	-15.0	Gate value below which a token is counted as deleted (<code>sigmoid_mask_scale / 2</code>).
<code>gate_layer_norm</code>	True	Apply LayerNorm to hidden states before computing the gate.
<code>use_gumbel_noise</code>	False	Add Gumbel noise to gate logits during training to encourage exploration.
<code>random_deletion_probability</code>	0.5	Used only by the random gate ablation.
<code>fixed_deletion_amount</code>	0.5	Used only by the fixed gate ablation.

Table 1: New configuration parameters added by `MrBertConfig`.

None of these exist in standard `BertConfig`. They expose the gate as a set of tunable hyperparameters without touching the core BERT architecture.

3 The Delete Gate

File: `mrbert/models/modeling_mrbert.py`, lines 160–249

3.1 The ScaledSigmoid Activation (Lines 160–169)

```
def forward(self, input):
    return self.sigmoid_mask_scale * torch.sigmoid(-input)
```

This maps the gate's raw logit to a value in the range $[-30, 0]$:

- **Large positive logit** $\rightarrow \sigma(-\text{large}) \approx 0 \rightarrow \text{gate value} \approx 0 \rightarrow \text{keep the token}$
- **Large negative logit** $\rightarrow \sigma(-\text{large_neg}) \approx 1 \rightarrow \text{gate value} \approx -30 \rightarrow \text{delete the token}$

The output is designed to be added directly to attention scores (see Section 4). A value of 0 has no effect; a value of -30 makes the softmax assign essentially zero attention weight to that token.

3.2 The SigmoidDeleteGate Module (Lines 178–249)

The gate is a small neural network sitting on top of one encoder layer:

```
hidden_state (768-dim)
  |
[LayerNorm]          <- normalizes the input for stable training
  |
Linear(768 -> 1)      <- learns which features predict
                      "this token is dispensable"
  |
ScaledSigmoid(-30)    <- maps the scalar logit to [-30, 0]
  |
gate_value per token
```

Initialization (lines 236–248): The linear layer is initialized with very small weights (`std=0.001`) and a large positive bias (10.0). This is critical. With `bias=10`:

$$\text{gate_value} = -30 \times \sigma(-10) \approx -30 \times 0.000045 \approx 0$$

All tokens start with a gate value near 0 (keep everything). The model has to actively learn to push tokens toward -30 . This prevents catastrophic deletion at the start of training before the model has learned anything useful.

4 Soft Deletion: Masking Attention

File: `mrbert/models/modeling_mrbert.py`, lines 393–441 (`MrBertSelfAttention.forward`)

Soft deletion is the default mode. Tokens are not removed from memory—they just become invisible to attention in subsequent layers.

4.1 How It Works (Lines 414–419)

```
if delete_gate_mask is not None:
    delete_gate_mask_expanded = delete_gate_mask.squeeze(-1) \
                                .unsqueeze(1).unsqueeze(1)
    attention_scores = attention_scores + delete_gate_mask_expanded
```

In standard BERT, attention scores are computed as $QK^T/\sqrt{d}$ and then passed to softmax. In MrBERT, before the softmax, we add the gate values to the attention scores along the key dimension.

Intuition: If token A has gate value -30 , then every other token’s attention score for attending to token A is reduced by 30. After softmax, token A receives essentially zero attention weight from every other token. It is still in the sequence but is completely ignored by all subsequent layers.

This is an elegant approach because:

- It requires no architectural surgery—just an additive bias before softmax.
- It is fully differentiable, so the gate can be trained end-to-end.
- The original BERT attention mechanism is untouched; the gate is purely additive.

4.2 Softmax1 Variant (Lines 422–425)

Optionally enabled via `use_softmax1=True`. Adds 1 to the softmax denominator, allowing attention weights to sum to less than 1. This lets the model express “I don’t want to attend to anything strongly” rather than being forced to distribute all attention mass across available tokens.

5 Hard Deletion: Physically Removing Tokens

File: `mrbert/models/modeling_mrbert.py`, lines 509–553 (`MrBertLayer`)

Hard deletion is an optional mode (`hard_delete=True`). Instead of masking attention, tokens below the deletion threshold are physically removed from the sequence tensor.

5.1 Step 1: Decide Which Tokens Survive (Lines 509–541)

```
keep_this = delete_gate_mask > deletion_threshold # True=keep, False
           =delete
```

A cumulative sum maps each surviving token to its new position in the shorter sequence. A new attention mask is computed for the shortened sequence.

5.2 Step 2: Gather Surviving Tokens (Lines 543–553)

```
new_hidden_states = torch.gather(hidden_states, 1, gather_indices)
```

`torch.gather` physically selects only the rows corresponding to kept tokens, reducing the sequence dimension from `seq_len` to `num_kept_tokens`.

Intuition: After hard deletion, layers 4–11 operate on a shorter sequence. Since attention is $O(n^2)$ in sequence length, removing 30% of tokens reduces attention compute in those layers by roughly $0.7^2 \approx 49\%$. This is the actual efficiency gain MrBERT is designed to achieve.

Why soft deletion is the default: Soft deletion is fully differentiable and easier to train. Hard deletion involves discrete decisions (keep/delete) which complicate gradients. In practice, soft deletion is used during training; hard deletion can be applied at inference time.

6 Protecting Special Tokens

File: `mrbert/models/modeling_mrbert.py`, lines 213–232 (`SigmoidDeleteGate.forward`)

Special tokens (`[CLS]`, `[SEP]`, `[PAD]`) receive hardcoded gate values:

- `[CLS]` and `[SEP]` are forced to gate value 0 (always kept). `[CLS]` carries the sequence representation used for classification; `[SEP]` marks sentence boundaries.
- `[PAD]` tokens are forced to gate value `-30` (always deleted). They carry no information and should never receive attention.

Only non-special tokens pass through the learned gate. This ensures the model cannot break itself by accidentally deleting structurally necessary tokens.

7 Where the Gate Sits in the Encoder

File: `mrbert/models/modeling_mrbert.py`, lines 644–647

The delete gate is attached to a single encoder layer, specified by `delete_gate_layer` (default: 3). The gate is computed after layer 3’s standard self-attention and FFN, using the output hidden states as input.

Why layer 3? After 3 layers, the model has built up enough contextual representation for the gate to make meaningful keep/delete decisions. Earlier layers may not have enough context; later layers have already done most of the work and there is less compute to save.

7.1 Forward Pass Gate Propagation (Lines 670–707)

During the forward pass, the encoder tracks the `delete_gate_mask` produced at layer 3. This mask is passed as input to every subsequent attention layer (layers 4–11), which apply the soft deletion bias. The gate is computed once and reused for the rest of the encoder.

8 Training: The Deletion Loss and PI Controller

File: `mrbert/training/train_mrbert.py`

Training MrBERT requires solving two objectives simultaneously:

1. Minimize the task loss (get good NLI accuracy).
2. Delete approximately the target fraction of tokens.

These two objectives can conflict—the easiest way to minimize task loss is to delete nothing. The deletion loss and PI controller are the mechanism that enforces the deletion target.

8.1 The Deletion Loss (Lines 347–387)

```
deletion_loss = delete_gate_output[non_pad_mask].mean()
```

This is simply the mean gate value across all non-pad tokens. Since gate values are in $[-30, 0]$, minimizing this loss pushes gate values toward -30 (more deletion). The task loss pushes the gate toward 0 (keep everything useful). The balance between these two forces determines the actual deletion rate.

8.2 The PI Controller (Lines 303–344)

A fixed `deletion_loss_weight` is fragile—too high and the gate collapses (deletes everything), too low and nothing gets deleted. Instead, the weight α is adjusted dynamically by a PI controller:

```
error = target_rate - actual_deletion_rate
p_acc = 0.9 * p_acc + 0.1 * kp * error      # proportional (EMA smoothed)
i_acc = i_acc + ki * error                  # integral (accumulates)
alpha = max(0.0, p_acc + i_acc)            # final weight
```

Intuition: If the actual deletion rate is below the target (not deleting enough), `error` is positive, so α increases, applying more deletion pressure. If the rate overshoots, `error` is negative, reducing α . The integral term prevents the rate from perpetually sitting slightly below target—it accumulates the error over time and applies a slow corrective push.

8.3 The Regularizer Delay (Lines 977–985)

```
if global_step >= args.regularizer_delay:
    loss = task_loss + current_alpha * deletion_loss
else:
    loss = task_loss
```

For the first `regularizer_delay` steps (default: 1000), only the task loss is optimized. This gives the model time to learn what makes a good NLI prediction before deletion pressure is applied. If deletion pressure starts immediately, the gate may learn to delete tokens before the model knows which ones are important.

8.4 Separate Learning Rate for the Gate (Lines 850–866)

```
optimizer = AdamW([
    {"params": other_params, "lr": args.learning_rate},          # 2e-5
    {"params": delete_gate_params, "lr": args.delete_gate_lr},  # 1e-4
])
```

The gate is a randomly initialized small network sitting on top of pretrained BERT weights. It needs to learn faster than the pretrained weights, which only need fine-tuning. A higher learning rate for gate parameters allows the gate to adapt quickly without destabilizing the pretrained representations.

9 Gate Variants

File: `mrbert/models/modeling_mrbert.py`, lines 178–346

Four gate implementations are available via `deletion_type` in the config:

Type	Class	Lines	Description
scaled_sigmoid	SigmoidDeleteGate	178–249	Main learnable gate. Trained end-to-end.
log_sigmoid	LogSigmoidDeleteGate	251–256	Same architecture, uses log sigmoid instead of scaled sigmoid.
random	RandomDeleteGate	259–299	Ablation: randomly deletes tokens at a configured rate.
fixed	FixedDeleteGate	301–346	Ablation: always deletes the same fixed fraction of non-special tokens.

Table 2: Gate variant implementations.

The random and fixed gates exist to answer the question: *does learning which tokens to delete actually matter, or is any deletion at the right rate equally good?*

10 What Stays the Same

Everything else in the BERT architecture is unchanged:

- The tokenizer and vocabulary
- The embedding layer

- The 12-layer transformer encoder structure
- Multi-head self-attention (except the additive mask)
- Feed-forward layers within each encoder layer
- Layer normalization and residual connections
- All task-specific heads (`MrBertForSequenceClassification`, `MrBertForMaskedLM`, etc.)—these are direct ports of the standard BERT task heads

The baseline for comparison is `modeling_bert.py`, which is a standard BERT implementation. Running with `-model-type BERT` uses this file directly, with no gate and no deletion, so results are directly comparable.

11 End-to-End Flow

Here is what happens to a single SNLI example (premise + hypothesis) during a MrBERT forward pass:

```

Input tokens: [CLS] The cat sat . [SEP] The animal rested . [SEP] [
  PAD] ...
      |
      BertEmbeddings (unchanged)
      |
      Encoder Layer 0 (standard BERT layer)
      Encoder Layer 1 (standard BERT layer)
      Encoder Layer 2 (standard BERT layer)
      |
      Encoder Layer 3 <- has delete gate
      +-- Standard BERT self-attention + FFN (produces hidden
states)
      +-- SigmoidDeleteGate:
          LayerNorm -> Linear(768->1) -> ScaledSigmoid
          +-----+
          | [CLS]  ->  0.0          (protected)      |
          | The    -> -2.1          (keep)            |
          | cat    -> -0.3          (keep)            |
          | sat    -> -18.4         (delete)           |
          | .      -> -24.1         (delete)           |
          | [SEP]  ->  0.0          (protected)      |
          | The    -> -19.2         (delete)           |
          | animal -> -1.1          (keep)            |
          | rested -> -0.8          (keep)            |
          | .      -> -22.3         (delete)           |
          | [SEP]  ->  0.0          (protected)      |
          | [PAD]  -> -30.0         (always deleted)  |
          +-----+
          | delete_gate_mask (passed to layers 4-11)
Encoder Layer 4 <- receives delete_gate_mask
  attention scores += delete_gate_mask
  -> deleted tokens receive ~0 attention weight
Encoder Layer 5 (same)
...
Encoder Layer 11 (same)
      |
      [CLS] hidden state -> classifier

```

```
-> entailment / neutral / contradiction
```

The key insight: layers 4–11 never “see” the deleted tokens in any meaningful sense. The gate has effectively compressed the sequence, and all subsequent computation operates on the surviving tokens only.

12 Key File Reference

Component	File	Lines
Config additions	mrbert/models/configuration_mrbert.py	68–71
ScaledSigmoid	mrbert/models/modeling_mrbert.py	160–169
SigmoidDeleteGate	mrbert/models/modeling_mrbert.py	178–249
Special token protection	mrbert/models/modeling_mrbert.py	213–232
Soft deletion	mrbert/models/modeling_mrbert.py	414–419
Hard deletion	mrbert/models/modeling_mrbert.py	509–553
Gate in encoder	mrbert/models/modeling_mrbert.py	644–647
PI controller	mrbert/training/train_mrbert.py	303–344
Deletion loss	mrbert/training/train_mrbert.py	347–387
Regularizer delay	mrbert/training/train_mrbert.py	977–985
Gate learning rate	mrbert/training/train_mrbert.py	850–866
Baseline BERT	mrbert/models/modeling_bert.py	—

Table 3: Key source files and line references.